

Практическая работа №2. Типы данных и ввод с клавиатуры

Дисциплина: Алгоритмизация и программирование

Инструменты: .NET SDK 10.0, VS Code + C# Dev Kit, Git Bash

Цели работы

- Изучить полную картину числовых типов C# — их диапазоны и когда какой использовать.
- Разобраться с типом `char` и экранированными последовательностями.
- Понять, что такое `var` и чем неявная типизация отличается от динамической.
- Научиться считывать данные с клавиатуры через `Console.ReadLine()`.
- Освоить три способа превратить строку в число: `Convert.ToInt32`, `int.Parse`, `int.TryParse` — и понять разницу между ними.

В ПР1 вы уже работали с переменными `int`, `double`, `string`, `bool`, константами и арифметикой. Здесь мы не повторяем это с нуля, а расширяем картину: полный список типов, ввод с клавиатуры и работа с "грязными" строковыми данными, которые вводит пользователь.

Теоретическая справка

Полная картина числовых типов

C# — **строго типизированный** язык: каждая переменная имеет фиксированный тип, и он не может "на лету" превратиться в другой (в отличие от Python, где `x = 5`, а потом `x = "текст"` — совершенно нормально).

Целочисленные типы:

Тип	Размер	Диапазон	Когда использовать
sbyte	1 байт	от -128 до 127	почти никогда, очень тесный диапазон
byte	1 байт	от 0 до 255	яркость пикселя, байты файла
short	2 байта	от -32 768 до 32 767	редко, когда важна экономия памяти
int	4 байта	от -2 147 483 648 до 2 147 483 647	основной тип для целых чисел — используем по умолчанию
long	8 байт	±9.2 квинтиллиона	очень большие числа (население Земли, микросекунды)

Дробные типы:

Тип	Точность	Когда использовать
float	~7 значащих цифр	почти никогда в учебных задачах, требует суффикс <code>f</code>
double	~15-16 значащих цифр	основной тип для дробных чисел — используем по умолчанию

decimal	28-29 значащих цифр, без погрешностей округления	деньги и финансовые расчёты — обязателен суффикс m
----------------	--	---

Почему для денег используют decimal, а не double? double хранит число в двоичном виде, и некоторые "простые" на вид десятичные дроби (например, 0.1) не могут быть представлены в двоичной системе абсолютно точно — накапливается крошечная погрешность. Для физических расчётов это не страшно, но при работе с деньгами копейка расхождения — уже баг. decimal устроен так, что подобных проблем не возникает.

Тип char

char хранит **один** символ в одинарных кавычках:

```
char grade = 'A';
char newLine = '\n'; // escape-последовательность — перенос строки
char tab = '\t'; // escape-последовательность — табуляция
char quote = '\''; // экранированная одинарная кавычка
```

Не путайте 'A' (char, одинарные кавычки) и "A" (string из одного символа, двойные кавычки) — это разные типы.

var — неявная типизация

```
var age = 20; // компилятор сам определит: int
var price = 99.9; // компилятор сам определит: double
var name = "Иван"; // компилятор сам определит: string
```

Важно: var — это **не** аналог динамической типизации Python. Тип всё равно определяется **один раз, во время компиляции**, просто вы не пишете его руками — компилятор смотрит на то, что справа от =, и подставляет тип сам. После объявления var age = 20; переменная age — это самый обычный int, и присвоить ей потом строку так же нельзя, как обычной int-переменной.

var удобен, когда тип и так очевиден из правой части или очень длинный для написания. Мы будем использовать его точно, а не везде подряд — в учебных целях явные типы (int, string) читаются понятнее.

Console.ReadLine и ввод с клавиатуры

```
Console.Write("Введите ваше имя: ");
string userName = Console.ReadLine();
Console.WriteLine($"Привет, {userName}!");
```

Console.ReadLine() всегда возвращает **string**, даже если пользователь ввёл число.

Если нужно число — строку придётся **преобразовать**.

Три способа превратить строку в число

```
string input = "25";

int a = Convert.ToInt32(input);    // способ 1
int b = int.Parse(input);          // способ 2
bool ok = int.TryParse(input, out int c); // способ 3
```

Способ	Что будет, если строка "кривая" (например, "abc")
Convert.ToInt32	Бросит исключение <code>FormatException</code> , программа упадёт с ошибкой
int.Parse	То же самое — бросит <code>FormatException</code>
int.TryParse	Не упадёт. Вернёт <code>false</code> в переменную <code>ok</code> , а в <code>c</code> запишет 0

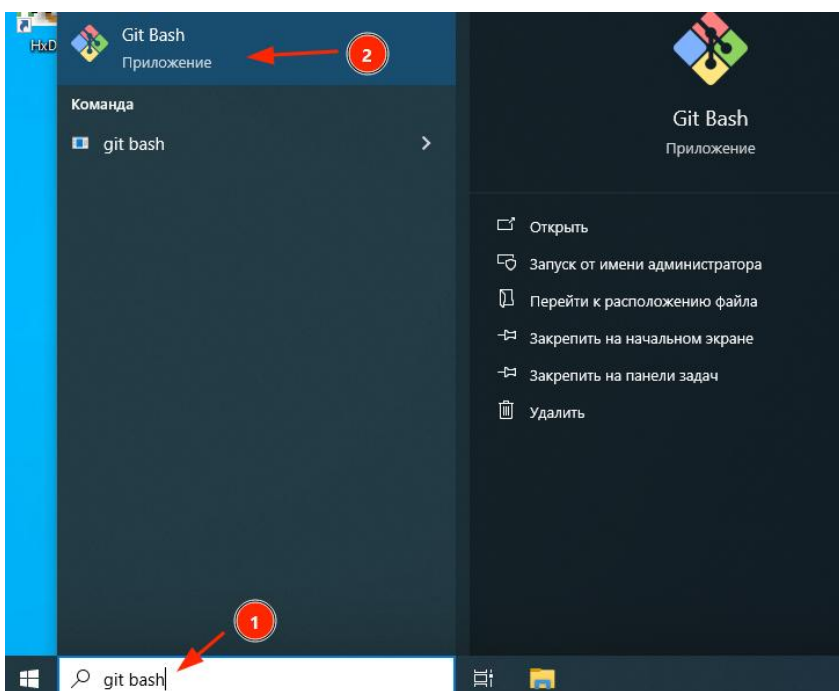
`TryParse` — единственный безопасный способ, когда вы не уверены, что введёт пользователь (а вы почти никогда не можете быть уверены). Именно поэтому `TryParse` и называется "try" — "попробовать", без гарантии успеха.

Реальную проверку `if (ok) { ... } else { ... }` мы напишем в **ПР4**, когда разберём условные конструкции. Сегодня наша задача — понять механику самого преобразования и увидеть, что результат `TryParse` — это `bool`, который вы обычно будете сразу использовать в условии.

Часть 1. Диапазоны типов

Шаг 0. Откройте Git Bash

В панели поиска Windows введите «**git bash**» и откройте программу:



Шаг 1. Создание проекта

1. В Git Bash выполните:

cd ~/Documents

2. Проверьте, где вы находитесь:

pwd

В терминале должен отображаться путь, заканчивающийся на **/Documents**:

```
denis@windows10 MINGW64 ~  
$ cd ~/Documents  
  
denis@windows10 MINGW64 ~/Documents  
$ pwd  
/c/Users/denis/Documents
```

Важно: после этого шага мы создадим отдельную папку проекта. В дальнейшем переходить обратно в **Documents** не нужно.

3. Выполните в терминале:

dotnet new console -n Lab02-Фамилия

Например:

dotnet new console -n Lab02-Leontev (укажите свою фамилию)

После этого появится сообщение:

```
Шаблон "Консольное приложение" успешно создан.  
  
Идет обработка действий после создания...  
Восстановление C:\Users\denis\Documents\Lab02-Leontev\Lab  
02-Leontev.csproj:  
Восстановление выполнено.
```

4. Теперь перейдите в созданный проект:

cd Lab02-Leontev (будет ваша фамилия)

```
denis@windows10 MINGW64 ~/Documents  
$ cd Lab02-Leontev/  
  
denis@windows10 MINGW64 ~/Documents/Lab02-Leontev
```

С этого момента вся практическая работа выполняется внутри этой папки.

5. Создайте папку **img** для сохранения будущих скриншотов.

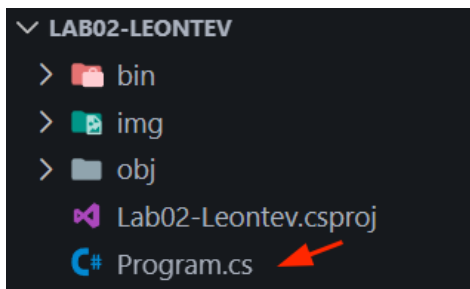
mkdir img

6. Далее введите в терминале:

code .

После выполнения команды откроется **VS Code** с проектом **Lab02-Фамилия**.

В левой части **VS Code** вы должны увидеть файл **Program.cs**, в нём мы будем писать весь следующий код:



Шаг 2. Печатаем границы типов

1. Удалите код внутри файла **Program.cs** и перепишите следующий код:

```
Console.WriteLine("Границы целочисленных типов");
Console.WriteLine($"byte: {byte.MinValue} .. {byte.MaxValue}");
Console.WriteLine($"short: {short.MinValue} .. {short.MaxValue}");
Console.WriteLine($"int: {int.MinValue} .. {int.MaxValue}");
Console.WriteLine($"long: {long.MinValue} .. {long.MaxValue}");

Console.WriteLine();
Console.WriteLine("Границы дробных типов");
Console.WriteLine($"float: {float.MinValue} .. {float.MaxValue}");
Console.WriteLine($"double: {double.MinValue} .. {double.MaxValue}");
Console.WriteLine($"decimal: {decimal.MinValue} .. {decimal.MaxValue}");
```

2. Откройте встроенный терминал в VS Code (Ctrl+J) и введите команду **dotnet run**

Шаг 3. Переполнение (overflow)

1. Добавьте далее в файл **Program.cs** следующий код:

```
Console.WriteLine();
Console.WriteLine("Переполнение byte");

byte maxByte = 255;
byte overflowed = (byte)(maxByte + 1);
Console.WriteLine($"255 + 1 для byte = {overflowed}");
```

2. Во встроенном терминале VS Code введите команду **dotnet run**

3. При запуске программы обратите внимание на результат — вместо 256 вы увидите 0. Это называется **переполнение (overflow)**: если результат не помещается в границы типа, значение "оборачивается" по кругу заново от минимума.

 **СКРИНШОТ** — вывод программы целиком (границы типов + переполнение).

Сохраните в папке **img** с именем **step3Lab02_ФИО.png**.

Шаг 4. Коммит

Во встроенном терминале VS Code введите команды:

git init -b main

git add .

git commit -m "Lab02: numeric type ranges and overflow"

Шаг 5. Перед заливкой на GitHub

Если вы работаете за общими компьютерами, возможны проблемы из-за разных пользователей. На **каждой** паре вам нужно будет выполнять следующие пункты:

1. Проверка текущего пользователя Git

Выполните в терминале:

git config user.name

git config user.email

Значения должны **совпадать с данными вашей учётной записи GitHub**.

При необходимости можно задать их вручную:

git config user.name "ваш_ник"

git config user.email "ваша_почта"

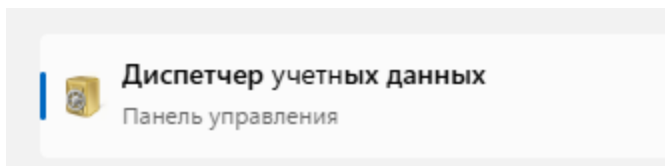
2. Вход в GitHub через браузер

Перед выполнением **git push** вы должны **выполнить вход в GitHub в браузере по умолчанию**, так как при отправке данных появится окно подтверждения авторизации.

3. Очистка старых учётных данных Windows

- Введите в поиске Windows:

Диспетчер учётных данных



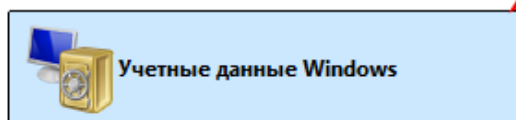
- Перейдите во вкладку **Учётные данные Windows**

Администрирование учётных данных

Просмотр и удаление сохранённых сведений о входе для веб-сайтов, подключённых приложений и сетей.



Учетные данные для Интернета



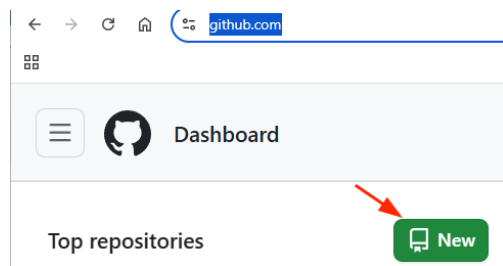
Учетные данные Windows

Удалите все сохранённые записи, связанные с **GitHub**

Шаг 5.5. Создайте репозиторий на GitHub

- Откройте [github](https://github.com) в браузере по умолчанию.

2. Нажмите New:



3. Введите:

Название репозитория:

Lab02-Фамилия

Например:

Lab02-Ivanov

Остальные параметры оставьте по умолчанию и нажмите **Create repository**

A screenshot of the GitHub 'Create new repository' form. The form is divided into two sections: '1 General' and '2 Configuration'. In the 'General' section, the 'Repository name' field is highlighted with a red box and a red arrow. The text 'Lab02-Leontev' is entered in this field, and a green checkmark with the text 'Lab02-Leontev is available.' is shown below it. The 'Owner' is set to 'Paltosik92'. The 'Description' field is empty. In the 'Configuration' section, 'Choose visibility' is set to 'Public', 'Add README' is 'Off', 'Add .gitignore' is 'No .gitignore', and 'Add license' is 'No license'. At the bottom of the form is a green button labeled 'Create repository' with a red arrow pointing to it.

Важно: репозиторий должен быть пустым. Мы уже создали проект и первый коммит на компьютере.

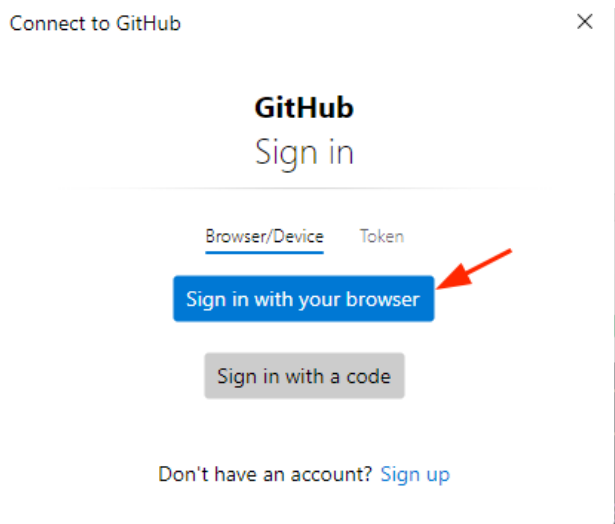
После создания **GitHub** покажет адрес репозитория и необходимые команды в самом конце для вставки в терминале:



4. Во встроенном терминале VS Code введите все эти команды (для вставки в терминале Git Bash используется сочетание клавиш **Shift + Insert**):

```
leontev@denis MINGW64 ~/Documents/Lab02-Leontev (main)
$ git remote add origin https://github.com/Paltosik92/Lab02-Leontev.git
git branch -M main
git push -u origin main
```

5. При выполнении последней команды и у вас появится окно подтверждения. Нажмите на синюю кнопку **Sign in with your browser**:



При удачной загрузке вы увидите успешный вывод в терминале VS Code:

```
git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 31, done.
Counting objects: 100% (31/31), done.
Delta compression using up to 16 threads
Compressing objects: 100% (27/27), done.
Writing objects: 100% (31/31), 87.19 KiB | 6.71 MiB/s, done.
Total 31 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/Paltosik92/Lab02-Leontev.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Решение проблем при push

Проверка привязки удалённого репозитория

Выполните команду:

git remote -v

Вы увидите список удалённых репозиториях.

Если у вас привязан **не тот репозиторий**, выполните перепривязку:

git remote remove origin

git remote set-url origin <новый_URL>

Часть 2. char и decimal на практике

Шаг 6. Работа с char

Добавьте далее в файл **Program.cs** следующий код:

```
Console.WriteLine();
Console.WriteLine("char");

char firstLetter = 'A';
char separator = '-';
int charAsNumber = firstLetter; // char можно неявно
превратить в int — это код символа в таблице Unicode
```



```
Console.WriteLine($"Символ: {firstLetter}, разделитель: {separator}");
Console.WriteLine($"Код символа '{firstLetter}' в Unicode: {charAsNumber}");
Console.WriteLine($"Табуляция:\tпосле таба");
Console.WriteLine($"Перенос:\nпосле переноса");
```

Шаг 7. decimal для денег

Добавьте далее в файл **Program.cs** следующий код

```
Console.WriteLine();
Console.WriteLine("decimal против double");

double priceDouble = 0.1 + 0.2;
decimal priceDecimal = 0.1m + 0.2m;

Console.WriteLine($"double: 0.1 + 0.2 = {priceDouble}");
Console.WriteLine($"decimal: 0.1 + 0.2 = {priceDecimal}");
```

Во встроенном терминале VS Code введите команду **dotnet run**

Внимательно сравните вывод — double покажет 0.300000000000000004 (или похожую погрешность в зависимости от форматирования), а decimal — ровно 0.3. Это и есть причина, по которой для денег используется decimal.

 **СКРИНШОТ** — вывод char + decimal против double. Сохраните в папку **img** с именем **step6Lab02_ФИО.png**.

Сделайте коммит в терминале VS Code:

git add .

git commit -m "Lab02: char and decimal precision"

git push

Часть 3. var на практике

Шаг 8. Проверяем, что var — это не "любой тип"

Добавьте далее в файл **Program.cs** следующий код:

```
Console.WriteLine();
Console.WriteLine("var");

var studentAge = 20;           // компилятор вывел int
var gpa = 4.75;                // компилятор вывел double
var fullName = "Смирнова А.С."; // компилятор вывел string

Console.WriteLine($"{{fullName}}, возраст {{studentAge}},
средний балл {{gpa}}");
```

Теперь попробуйте добавить строку `studentAge = "двадцать";` сразу после объявления и во встроенном терминале VS Code введите команду **dotnet run**

Компилятор выдаст ошибку компиляции: **Cannot implicitly convert type 'string' to 'int'**. **Уберите** эту строку обратно — она нужна была только чтобы увидеть ошибку своими глазами.

Это доказывает: `var studentAge = 20;` — это ровно то же самое, что `int studentAge = 20;`, просто тип не написан явно, а вычислен компилятором один раз при компиляции. Присвоить туда `string` нельзя точно так же, как обычному `int`.

Шаг 9. Коммит

Сделайте коммит в терминале VS Code:

git add .

git commit -m "Lab02: var keyword"

git push

Часть 4. Ввод с клавиатуры

Шаг 10. Первый `Console.ReadLine`

Добавьте далее в файл **Program.cs** следующий код:

```
Console.WriteLine();
Console.WriteLine("Ввод текста");

Console.Write("Введите ваше имя: ");
string enteredName = Console.ReadLine();

Console.Write("Введите название вашей группы: ");
string enteredGroup = Console.ReadLine();

Console.WriteLine($"Здравствуйте, {enteredName} из группы {enteredGroup}!");
```

Во встроенном терминале VS Code введите команду **dotnet run** и введите своё имя и группу, когда программа попросит.

Обратите внимание: `Console.Write` (без `Line`) не переносит строку после вывода — поэтому текст, который вводит пользователь, окажется на той же строке, что и приглашение "Введите ваше имя: ". `Console.WriteLine` — с переносом.

Шаг 11. Коммит

Сделайте коммит в терминале VS Code:

git add .

git commit -m "Lab02: keyboard input with ReadLine"

git push

Часть 5. Преобразование строки в число

Шаг 12. Convert.ToInt32 и int.Parse

Добавьте далее в файл **Program.cs** следующий код:

```
Console.WriteLine();
Console.WriteLine("Ввод чисел: Convert и Parse");

Console.Write("Введите ваш год рождения: ");
string birthYearInput = Console.ReadLine();

int birthYearConvert = Convert.ToInt32(birthYearInput);
int birthYearParse = int.Parse(birthYearInput);

Console.WriteLine($"Convert.ToInt32: {birthYearConvert}");
Console.WriteLine($"int.Parse: {birthYearParse}");
Console.WriteLine($"В 2030 году вам будет: {2030 - birthYearConvert} лет");
```

Во встроенном терминале VS Code введите команду **dotnet run** и введите год рождения **числом**, например 2006. Всё должно отработать без ошибок.

Теперь запустите ещё раз и введите **не число**, например двухтысячный. Программа "упадёт" с необработанным исключением `FormatException` — вы увидите красный текст с описанием ошибки в терминале. Это нормально, так и должно произойти — сейчас мы специально смотрим, что случается с "хрупким" кодом, который не проверяет ввод.

 **СКРИНШОТ** — вывод терминала с ошибкой `FormatException` после ввода нечислового значения. Сохраните в папке **img** с именем **step12Lab02_ФИО.png**.

Шаг 13. int.TryParse — безопасный способ

Добавьте далее в файл **Program.cs** следующий код:

```
Console.WriteLine();
Console.WriteLine("Ввод чисел: TryParse");

Console.Write("Введите количество прочитанных книг за семестр: ");
string booksInput = Console.ReadLine();

bool wasSuccessful = int.TryParse(booksInput, out int booksCount);

Console.WriteLine($"Удалось преобразовать: {wasSuccessful}");
Console.WriteLine($"Значение переменной booksCount: {booksCount}");
```

Запустите программу дважды: один раз введите число (например 5), второй раз — не число (например много).

Сравните поведение с предыдущим шагом: во второй раз программа **не упала**, а просто вывела `wasSuccessful = False` и `booksCount = 0`.

Обратите внимание на конструкцию `out int booksCount` — она объявляет переменную `booksCount` **прямо внутри вызова метода** и метод сам заполняет её значением. Это специфический синтаксис C#, с параметрами `out` мы ещё встретимся в ПР8.

Пока **`wasSuccessful`** — это просто переменная типа `bool`, которую мы выводим на экран. Начиная с ПР4 (условные конструкции) мы будем писать `if (wasSuccessful) { ... } else { Console.WriteLine("Ошибка ввода!"); }` — то есть **реагировать** на результат проверки, а не просто печатать его.

Шаг 14. Коммит

Сделайте коммит в терминале VS Code:

`git add .`

`git commit -m "Lab02: string to number conversion, Parse vs TryParse"`

`git push`

Часть 6. Итоговая практика: мини-анкета

Шаг 15. Собираем анкету студента

Добавьте в конец **`Program.cs`** блок, который **последовательно спрашивает** у пользователя через **`Console.ReadLine()`**:

1. Имя и фамилию (`string`).
2. Название группы (`string`).
3. Год рождения (`int`, через `int.Parse` или `Convert.ToInt32`).
4. Средний балл за прошлый семестр (`double`, через `double.Parse`).
5. Любимую букву алфавита (`char` — подсказка: `Console.ReadLine()[0]` берёт первый символ строки).

А затем **выводит** аккуратную сводку по всем введенным данным, используя интерполяцию строк, и **вычисляет**:

- Сколько лет исполнится студенту в 2030 году.
- Является ли средний балл `>= 4.0` (результат сравнения выведите как есть — это `bool`, отдельный тип, полноценно работать с ним в условиях начнём в ПР4).

Пример работы:

```
Введите имя и фамилию: Иванов Иван
Введите группу: ИСП-251
Введите год рождения: 2009
Введите средний балл: 4.6
Введите любимую букву: Я

    Анкета
Иванов Иван, группа ИС-21
Год рождения: 2009 (в 2030 будет 21 год)
Средний балл: 4.6
Балл >= 4.0: True
Любимая буква: Я
```

Во встроенном терминале VS Code введите команду **dotnet run**

 **СКРИНШОТ** — полный вывод программы «Анкета» с введенными вами данными. Сохраните в папке **img** с именем **step15Lab02_ФИО.png**.

Шаг 16. Финальный коммит

Сделайте коммит в терминале VS Code:

git add .

git commit -m "Lab02 final: student questionnaire with input"

git push

Самостоятельные задания

Задание 1. Калькулятор ИМТ ★

Добавьте в конец **Program.cs** блок, который:

1. Спрашивает у пользователя рост в метрах (double, например 1.75).
2. Спрашивает вес в килограммах (double, например 68.5).
3. Считает индекс массы тела по формуле $ИМТ = \text{вес} / (\text{рост} * \text{рост})$.
4. Выводит результат с округлением через :F2 в интерполяции:
`Console.WriteLine($"ИМТ: {bmi:F2}");`

Сделайте коммит в терминале VS Code:

git add .

git commit -m "Lab02 task 1: BMI calculator"

git push

Задание 2. Разбор ФИО через char ★★

Добавьте в конец **Program.cs** блок, который:

1. Спрашивает фамилию и имя **отдельными** запросами (`Console.ReadLine()` дважды).
2. Берёт **первую букву** имени как `char` (подсказка: индексация строки — `name[0]`).
3. Выводит результат в формате Фамилия И. (например, Смирнова А.).

Сделайте коммит в терминале VS Code:

git add .

git commit -m "Lab02 task 2: name initials with char"

git push

Задание 3. TryParse на трёх типах ★★★

Добавьте в конец **Program.cs** блок, который спрашивает у пользователя **три** значения и для **каждого** использует `TryParse`, выводя и результат преобразования, и флаг успеха:

1. Целое число через `int.TryParse`.
2. Дробное число через `double.TryParse`.
3. Дату через `DateTime.TryParse` (изучите синтаксис самостоятельно по документации Microsoft — принцип работы такой же, как у `int.TryParse`, только результат типа `DateTime`, а вводить дату нужно в формате дд.мм.гггг, например 15.03.2006).

Выведите для каждого из трёх ввода понятную строку: успешно ли преобразование и какое значение получилось.

Сделайте коммит в терминале VS Code:

git add .

git commit -m "Lab02 task 3: TryParse for int, double, DateTime"

git push

Контрольные вопросы

Во встроенном терминале VS Code введите команду:

touch README.md

README.md — текстовый файл вашего репозитория. В конце работы сюда будут добавлены ответы на контрольные вопросы.

Откройте **README.md** и добавьте раздел «**ПР2. Ответы на контрольные вопросы**». Ответьте на:

1. Почему для хранения денег используют **decimal**, а не **double**? Что может пойти не так с **double**?
2. Что произойдёт, если целочисленный тип (например, **byte**) переполнится — то есть значение выйдет за границы диапазона?
3. Чем **var** отличается от динамической типизации в Python? В какой момент компилятор определяет тип переменной, объявленной через **var**?
4. Какой тип данных **всегда** возвращает **Console.ReadLine()**, независимо от того, что ввёл пользователь?
5. Чем **int.Parse** отличается от **int.TryParse** в поведении при некорректном вводе (например, буквы вместо цифр)?
6. Что означает **false**, возвращённый из **int.TryParse**, и что при этом попадёт в переменную, переданную через **out**?
7. Приведите пример, когда стоит использовать **TryParse**, а не **Parse** — и объясните, почему.

Сделайте коммит в терминале VS Code:

git add .

git commit -m "Lab02 add Readme.md and completed"

git push